

HERIOT-WATT UNIVERSITY

MASTERS THESIS

An Improved Syntax for C++ Smart Pointers

Author:

Xavier MITAULT

Supervisor:

Dr. Adam SAMPSON

*A thesis submitted in fulfilment of the requirements
for the degree of MSc.*

in the

School of Mathematical and Computer Sciences

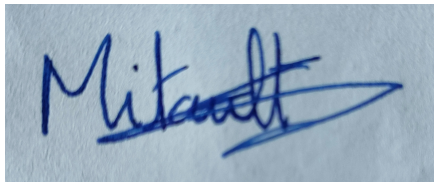
August 2025



Declaration of Authorship

I, Xavier MITAULT, declare that this thesis titled, 'An Improved Syntax for C++ Smart Pointers' and the work presented in it is my own. I confirm that this work submitted for assessment is my own and is expressed in my own words. Any uses made within it of the works of other authors in any form (e.g., ideas, equations, figures, text, tables, programs) are properly acknowledged at any point of their use. A list of the references employed is included.

Signed:

A handwritten signature in blue ink, appearing to read 'Mitaault', with a long horizontal flourish extending to the right.

Date:

Aug 11, 2025

Abstract

C++ is a programming language that combines high-level features from multiple paradigms with low-level access to the hardware. In C++, smart pointers are a way to manage memory allocation and automatic de-allocation. The usage of smart pointers is preferred over plain allocation and de-allocation (`new` and `delete`) for memory management in C++ from C++11. To use these smart pointers, the syntax is the same as any other class templates: `std::unique_pointer<underlying_type>` (for a “unique” smart pointer). Software using a lot of smart pointers either use a “`typedef`” to bind the smart pointer and the underlying type to a new type, or use the plain syntax everywhere. These two solutions are cumbersome. The syntax for a raw pointer type is easier to use as a type than a smart pointer. Even with the introduction of the “`auto`” keyword to let the compiler deduce the type, the full type needs to be written in a function argument, or declared somewhere when initialization is not possible (and the compiler will not be able to infer the type). This dissertation designs a new syntax for C++ smart pointers based on the background literatures and the syntax of a raw pointer: “`underlying_type %`”. We then implement the syntax on an existing C++ compiler: Clang, by first documenting how templates are being created in the compiler (there not much documentation on this part), then allowing our syntax next to the parts that parses raw pointer, and finally replicating a template when creating the type. The implementation works on real world project and does not break compatibility. Finally, we evaluate the designed syntax against the current one using the Halstead Metrics: Counting the usage of specific language syntax operators and operands to evaluate the code complexity. We find that the designed syntax is more efficient to use. In terms of performance, we find that there is no performance overhead on the output binary when using our modified version of Clang. There is an overhead on using our modified Clang version when compiling code using our syntax, but it is less than 0.1 seconds for each files.

Acknowledgements

I would like to thank the following people for helping with this dissertation:

Dr. Adam Sampson, my supervisor, for all the insights and help he gave me during the Research Project and the Dissertation. He was there every week to answers the issues I had. Mr Florent Mitault, my father, and Ms Aurélie Mitault, my sister, for the support.

I would also like to thank the following university for providing any support:

Heriot-Watt University for learning quality and for the facilities on campus that helped me focus on this dissertation. The EPITECH School for giving me the opportunity to study abroad for a whole year.

Contents

Declaration of Authorship	i
Abstract	ii
Acknowledgements	iii
Contents	iv
List of Figures	vii
List of Tables	viii
Abbreviations	ix
Symbols	x
1 Introduction	1
1.1 Context	1
1.1.1 Previous Work	2
1.1.2 Smart Pointer Syntax vs Raw Pointer Syntax	3
1.1.3 Motivation	3
1.2 Aim and Objectives	3
1.3 Organization	4
2 Background	5
2.1 The C++ Language	5
2.2 The Choice of Compiler	8
2.2.1 GNU and GCC	8
2.2.2 LLVM and Clang	9
2.2.3 MSVC	10
2.2.4 The Choice	11
2.3 Smart Pointers	11
2.4 Lifetime of a Variable	13
2.5 C++ New Syntax Projects Alternative: Cpp2 and cppfront	14
2.6 Evaluating a Syntax	16
2.6.1 Cyclomatic Complexity	16
2.6.2 The Halstead Metrics	16

2.7	Guidelines for a New Syntax	18
3	Specifications	19
3.1	Syntax Design Specifications	19
3.1.1	SDS1	19
3.1.2	SDS2	19
3.1.3	SDS3	19
3.1.4	SDS4	20
3.2	Technical Specifications	20
3.2.1	TS1	20
3.2.2	TS2	20
3.2.3	TS3	20
3.2.4	TS4	20
4	Implementation	21
4.1	Understanding the Clang Compiler Internals	21
4.1.1	Overview of the Clang Parser	22
4.1.2	How Templates are processed in the Clang Parser	22
4.1.3	Where a Raw Pointers Gets Parsed	24
4.1.4	The Clang Driver Command Line Flags	25
4.2	Modifying Clang	26
4.2.1	Adding a New Clang Command Line Flag	26
4.2.2	Adding a New DeclaratorChunk	26
4.2.3	Accept the New Syntax	27
4.2.4	Modify the Type	27
4.3	Final Implementation	27
4.4	Alternative Approaches	28
4.4.1	Why Not Changing the Tokens in the Lexer ?	28
4.4.2	Why Not Changing the AST Type during Code Generation ?	29
4.4.3	Why Using a Command Line Flag instead of a pragma directive ?	29
5	Evaluation	30
5.1	Design Evaluation - Halstead Metrics	30
5.1.1	Example 1	30
5.1.2	Example 2	32
5.1.3	Example 3	33
5.1.4	Summary	35
5.2	Technical evaluation	35
5.2.1	Performance of the Compiler	36
5.2.2	Performance of the Output Binary	37
5.3	Clang Tests	37
6	Conclusion	39
6.1	Summary of the Final Requirements	39
6.1.1	Syntax Design Specifications	39
6.1.1.1	SDS1	39
6.1.1.2	SDS2	40
6.1.1.3	SDS3	40

6.1.1.4	SDS4	40
6.1.2	Technical Specifications	40
6.1.2.1	TS1	40
6.1.2.2	TS2	41
6.1.2.3	TS3	41
6.1.2.4	TS4	42
6.2	Limits and Future Works	42
6.3	In Closing	43
 A External Links		44
 B Quick Sort Code		45
B.1	Current Syntax	45
B.2	New Syntax	46
 C Script Performance of the Compiler		47
C.1	Script Performance of the Compiler	47
 D Script Performance of the Output Binary		50
D.1	Script Performance of the Output Binary	50
D.2	Sort Algo New Syntax	51
D.3	Sort Algo Current Syntax	52
 References		54

List of Figures

4.1	Understanding the core components of the Clang compiler driver, Slide from van Haastregt and Stulova [2019]	21
4.2	Template Calls	23

List of Tables

5.1	Halstead Metrics for 5.1, New Syntax	31
5.2	Halstead Metrics for 5.1, Current Syntax	31
5.3	Halstead Metrics for 5.2, New Syntax	32
5.4	Halstead Metrics for 5.2, Current Syntax	33
5.5	Halstead Metrics for 5.3, New Syntax	34
5.6	Halstead Metrics for 5.3, Current Syntax	34

Abbreviations

AST	A bstract S yntax T ree
GNU	G NU's N ot U nix!
GCC	G NU C ompiler C ollection
MSVC	M icrosoft V isual C ++
IDE	I ntegrated D evelopment E nvironment

Symbols

Halstead Metrics

η_1	Number of distinct operators
η_2	Number of distinct operands.
N_1	Total number of operators.
N_2	Total number of operands.
η	Program Vocabulary
N	Program Length
$\hat{N}$	Calculated Estimated Program Length
V	Volume
D	Difficulty
E	Effort
T	Time Required to Program
B	Number of Delivered Bugs

Chapter 1

Introduction

C++ is a programming language that combines high-level features from multiple paradigms with low-level access to the hardware. C++ started in 1979 as a tool that added classes on top of C. Later, when the compiler front-end got modified, the design choice stuck to allow any valid C code as well as the new features of C++. Nearly any C code can be compliant with C++, with the exception of certain type warnings that become errors in C++, and new keywords that C++ added ([Stroustrup \[1993\]](#)).

In C++, smart pointers are a way to manage dynamic memory allocation and automatic de-allocation. To use this smart pointers, the syntax is the same as any other class templates: `std::unique_pointer<underlying_type>`.

The usage of smart pointers is preferred over plain allocation and de-allocation using `new` and `delete` ([Dmitrovic \[2020\]](#), [CppReference \[2024\]](#)). Software using a lot of smart pointers either use a `typedef` to bind the smart pointer and the underlying type to a new type, or use the plain syntax everywhere. These two solutions are cumbersome.

In this dissertation, we want to create a new syntax improving the usage of smart pointer for the programmers.

1.1 Context

In C and C++, when creating data of dynamic size, the program asks the operating system for a space in memory. This space will be used to store the data. This routine

is called memory allocation.

In C, this is typically done with “`malloc`” to allocate and “`free`” to de-allocate ([ISO \[2001\]](#)). This syntax gives back a pointer to the new allocated memory. A pointer is the address where the data are. You can access/edit the data using the pointer. A new syntax was added to C++ in 1985 to allocate an object and call the constructor at the same time: “`new`”, and “`delete`” to call the destructor and free the resource.

But, as with the C syntax, the programmer needs to de-allocate the memory after use. If the variable is forgotten, the memory will not be claimed until the end of the program ([Free Software Foundation \[2023\]](#)). In 1998, a new class was made available in the standard. The “`std::auto_ptr`” was added to delete the allocated object when the “`auto_ptr`” variable goes out of scope ([ISO \[1999\]](#)).

One problem remains: pointer access. If at some point, two pointers get the same address, modifying the underlying data of one pointer “modifies” the data of the other one. In C++ a new way to specify data ownership and value moves has been implemented. So in C++11, the smart pointers “`std::unique_ptr`” and “`std::shared_ptr`” entered into the standard library ([ISO \[2011\]](#)). They added pointer “ownership”, preventing a pointer to be owned by two smart pointers. They also integrated “`std::move`” to move a pointer address from one smart pointer to another one. The previous “`std::auto_ptr`” became deprecated because “`std::unique_ptr`” was a direct replacement. Shared smart pointers adds thread safety access to the underlying data.

in C++17 “`std::auto_ptr`” was removed ([ISO \[2018\]](#)).

In the new books teaching C++ [Dmitrovic \[2020\]](#) and on the [CppReference \[2024\]](#) for the keyword “`new`”, it is mentioned to switch to a smart pointer instead of a raw pointer.

1.1.1 Previous Work

Two previous works on modifying the syntax for C++ are known by the author. But one being from the 1997 ([Hamilton \[1997\]](#)), the tools used are no longer existent, and the other one is a new skin syntax for C++ ([Sutter \[2025\]](#)).

This dissertation is different: it aims to add only the syntax for smart pointer without including breaking change to the C++ standard.

1.1.2 Smart Pointer Syntax vs Raw Pointer Syntax

The syntax for a raw pointer type with the underlying type is “`T*`” (“`T`” representing an arbitrary type).

The syntax for a smart pointer from the standard (unique pointer) with the underlying type is “`std::unique_ptr<T>`”, and for a shared pointer from the standard, it is “`std::shared_ptr<T>`”.

1.1.3 Motivation

Even with the introduction of the “`auto`” keyword for type inference when declaring and initializing a type, at some point we still need to pass it as an argument, or declare it somewhere where we can’t initialize it.

Ditching raw pointer and using smart pointers is recommended in modern C++ ([1.1](#)). This feature is also one the main advantages of C++ over C. They are widely used, but no custom syntax helps their usage. This dissertation aims to create a syntax for smart pointers, letting them being integrated in the language. These changes aims to help developer use them more efficiently in the code.

1.2 Aim and Objectives

This dissertation aims to help the efficiency of developers when using smart pointers. We will design a new syntax for C++ smart pointers thus being more efficiently to use, and integrating it into an existing compiler: Clang.

The steps are to:

- Design a syntax for C++ smart pointers with the help of different syntax from other programming languages and research;
- Modify an existing C++ compiler, Clang, to parse and compile this syntax;
- Migrate an existing code base using smart pointers to the new syntax;
- Discuss the Halstead complexity difference between the syntax proposed in this dissertation and the standard one. Evaluate the impact on the compilation speed;

1.3 Organization

The rest of the document is organised as follow:

- Chapter 2 provides background on the subject.
- Chapter 3 shows the specifications and requirements of the desired C++ compiler and the improved syntax.
- Chapter 4 presents the implementation and redesign steps to modify the clang compiler.
- Chapter 5 presents the evaluation of the modified compiler and the improved syntax.
- Chapter 6 provides the conclusion of this dissertation.

Chapter 2

Background

2.1 The C++ Language

C++ was first implemented on top of the C language with classes ([Stroustrup \[1993\]](#)). When first released in 1979, the new features added that did not exist in C were ([CpReference \[2024\]](#)):

- Classes
- Member function: Function inside classes, also called methods.
- Derived classes: Any class can take the properties of another class, also called inheritance.
- Private and public access for methods and attributes.
- Friends: Allow access to private methods or attribute from another class.
- Default arguments: Default value for methods or function parameters.
- Inline functions: Copy the functions to where it is called instead of calling it.
- Overloaded assignment operator: The developer can implement the logic behind the assignment operator `=`.
- Constructor and Destructor: The developer can implement the initialization / de-initialization logic of a class.

- Sugar Syntax for function/methods with no arguments: `f()` is the same as `f(void)`.

It then added more features in 1985 for Cfront 1.0 ([CppReference \[2024\]](#)):

- Virtual functions: Specify the method signature to implement if a class inherits from the parent. This methods is not implemented in the parent.
- More operator overloading: The developer can implement the logic for the operators.
- Reference: Represent the address of a variable. But the difference with C pointers is the fact that references cannot be null (contains no address), and need to be always initialized.
- New: Allocate class variable on the heap, and call its constructor. Before, the developer needed to call C `malloc` function, then the constructor.
- Delete: Call the the destructor of the class then de-allocate the variable. Before, the developer needed to manually run the de-constructor, then the C `free` function.
- Scope resolution operator: Access specific methods function from a different scope. For example: `TheScope::TheMethod`.
- String: A class specific for handling characters strings.
- Complex numbers: A class to handle complex numbers.
- IO Stream: Input operations are appended and the end of the chain fo stream append, flush it.

And in 1989, Cfront 2.0 is released with ([CppReference \[2024\]](#)):

- Multiple inheritance: A class can inherit from multiple parents.
- Pointers to members: Highter order function. Let the developper get a reference to a methods, and call the referenced methods.
- Protected access: Private access to methods / attributes but let the childrens of this class access this protected member too.

- Abstract classes: A class that cannot be instantiated on its own. A children need to implement the virtual methods first, for the children to be able to be instantiated.
- Static members: Members that are the same data for every instance of the class.
- Const-qualified member function: Methods that cannot modify the members of the class.
- New and Delete overloading: The developer can implement the logic for new and delete for a specific class.
- IO manipulators: Change how the stream behave when receiving some values.

At this point, the language changed from a **preprocessor** to C (and outputting C to a C compiler) to a new front-end that used the GNU Compiler Collection (GCC) (2.2.1). The language changed to become a standard, with the first standard being C++98 in 1998, allowing other compiler implementations (ISO [1999]).

The C++ language now combines high-level features from multiple paradigms with low-level access to the hardware. Theses features are:

- Procedural: Subdivides tasks in procedures (function).
- Object Oriented: Divide tasks into individual units (class).
 - Encapsulation: Hide internal procedures and variables.
 - Inheritance: Take functionality of the parent class.
 - Multiple Inheritance: Take the functionalities of the parents class.
 - Polymorphism: Let one method name be used with different parameters.
- Higher Order Functions: possibility to give a functions as a arguments (to methods and functions).
- Statically Typed: Each variable has a type and it is checked by the compiler.
- Generic:
 - Templates: program an algorithm without specifying the value type.

The language being more high-level than C, it allows programmers to focus on code logic instead of doing everything by hand. But still allow low-level construct to find the best performance tuning where needed. For these reasons, the language is picked in real-world application: in Game Development (Used for Unreal Engine, Godot), in modern Browser (Firefox, Chromium), Machine Learning (Tensorflow), Search Engine (Google), Compiler (GCC, Clang).

2.2 The Choice of Compiler

To integrate our new syntax, and evaluate it in real situation, It is necessary to modify an existing compiler.

There is a 3 main compilers that can compile C++:

1. GCC
2. Clang
3. MSVC

2.2.1 GNU and GCC

GNU's Not Unix! (recursive acronym: GNU) is an operating system using software with a free license. The majority of the program used in this operating system is developed in collaboration with GNU. This programs are called GNU packages ([GNU \[2025\]](#)). Some of this packages are well used on other Linux distributions, for example:

- Bash: shell.
- gdb: program debugger.
- gcc: compiler for C, C++ (since 1987 ([GCC \[2025\]](#))), Objective-C, Fortran, Ada, Go, D, Modula-2.
- grep: searching utility for characters in files.
- make: tool to generates files based on rules.

- emacs, nano: text editors.
- GIMP: image editing tool.

GCC, or the GNU Compiler Collection was originally a compiler for the GNU operating system. It is now a collection of compiler for C, C++, Objective-C, Fortran, Ada, Go, D, Modula-2. GCC's goals are the usage of free software and 'high quality releases' (GCC [2025]). The C compiler was originally written in C, but then was migrated to C++. The change from C to C++ in GCC was incremental (Jacobson [2013]). It is written that from 2013, people adding features to GCC can do so using C++ if they want to, but it is not mandatory to use special feature from C++. It lets the GCC compiler in a state that is half-C++ half-C.

When giving a file to compile, GCC calls the language front-end once. This front-end is in charge of converting its representation of the source code to a intermediate representation that is independant from the original programming language. In GCC, this representation is called GIMPLE (GCC [2025]).

2.2.2 LLVM and Clang

LLVM started as a research project to compile any language. They developed an Internal Representation (IR) of a program. This IR is then processed by the LLVM back-end to compile it to a binary. That allows some programming languages to focus on implementing the front-end parsing of the language and create the IR for LLVM to compile it to a binary. LLVM takes care of the different architectures of the machine and optimizing the final binary (LLVM [2025]).

They now have more projects attached to it:

- Clang: a front-end parser for the C language family (C, C++, Objective C/C++, OpenCL, and CUDA).
- Clang-format: a tool integrated in clang to format code using predefined standard.
- LLDB: a program debugger that takes advantages of some LLVM program specifics. LLDB claims to be more efficient at loading symbols than GDB (GCC debugger).
- Libc++: an implementation of the C++ standard libraries.

- Libc: an implementation of the C standard libraries.
- Compiler-rt: implementation of low-level routines not supported for a machine architecture. Include runtime libraries used by LLVM programs.
- MLIR: a project to integrate multiple compilers for one output program.
- OpenMP: implementation of the OpenMP standard. A standard for parallel functions on a shared memory machine.
- Polly: implements some optimization with cache-locality, auto-parallelism and vectorization.
- Libclc: implementation of the OpenCL standard to use the Graphical Processing Unit (GPU).
- Klee: Run checks to prove code correctness on a virtual machine and produce test case when it fails.
- LLD: a linker for binaries. It claims to run faster than existing ones.
- BOLT: binaries post-link optimizer.

Clang is a front-end compiler for the C language family (C, C++, Objective C/C++, OpenCL, and CUDA) that is using the LLVM back-end. This means that it can parse C++ and passes the result to the LLVM tool-chains to compile it. The goal of Clang is to make better diagnostics, and be easy to develop and maintain ([Clang \[2025\]](#)). Clang's goal is also to document every feature added. Compatibility with the most used extension of GCC is also targeted.

Clang is used in large code-bases. Chromium default compiler is Clang. In fact, the main website of Chromium specify that Clang is the only official compiler ([for Computing Machinery \[2025\]](#)).

2.2.3 MSVC

The Microsoft Visual C++ compiler is the compiler used in the Visual Studio Integrated Development Environment (IDE). It primarily targets the Windows operating system.

Although the Standard Library code is open-source, the compiler code is not available, thus this compiler is not a suitable choice for this dissertation.

2.2.4 The Choice

GCC and Clang are two compilers for C++ that we could use to integrate our new syntax. Because of clang's goal about maintainability, and seeing that the code source was first written in C++, we will be modifying the clang front-end to implement the new syntax described in this dissertation.

2.3 Smart Pointers

In C++, the RAII principle is used. Resource Acquisition Is Initialization (RAII) means that the lifetime of the variable is bound with the lifetime of a resource.

Here is an example of opening a file for read only, the file (the resource) is closed when the variable goes out of scope.

```
1 #include <fstream>
2 // ...
3 {
4     std::ifstream myfile("log.txt");
5 }
6 // myfile is closed when the variable goes out of scope
```

LISTING 2.1: C++ RAII file example

Smart pointers use the same principle but for memory. They hold the allocated memory in an object. When this object is deleted, it frees the memory resource. They also provide checks for their underlying object: if they change of owner with C++ move semantics ([ISO \[2011\]](#)).

There is 3 types of smart pointer:

- Unique pointer ensures that there is only one reference to the memory address being hold, and when the unique pointer goes out of scope, the specific destructor is called.

- Shared pointer keeps a counter of all shared pointer that holds the same reference, when a shared pointer goes out of scope it checks if there is still other shared pointer referencing the holding address. If there is none, it calls the destructor.
- Weak pointer are objects that can create a shared pointer when they want to access the value being hold. Weak pointer are created from an original shared pointer.

Although smart pointer are used and recommended, performance overhead have been discussed. The standard says that there is little to no overhead (ISO [2011]) when using “`std::unique_ptr`” for creating / accessing the underlying pointer. In Carruth [2019] conference, it is shown that, the overhead is when the smart pointer is passed as a argument to a function. The conclusion of the conference is that for each abstraction, the benefits must outweigh the cons, but we need to know the cons to choose appropriately. For “`std::shared_ptr`” there is an overhead (in addition to the one of the unique pointer) when the constructor is called, to increment a counter, and when the destructor is called to check if it is the last reference to the pointer (ISO [2011]).

The current syntax for Unique and Shared pointers is:

```

1 #include <memory>
2 // ...
3 std::shared_ptr<int> a = std::make_shared<int>(5);
4 {
5     std::shared_ptr<int> b = a; // holding the same memory resource
6     std::unique_ptr<int> c = std::make_unique<int>(4);
7 }
8 // 'c''s value will be destroyed,
9 // the 'a' shared pointer being still in the scope, 'b' will not destroy its
   value.
```

LISTING 2.2: C++ Smart Pointer Syntax

In the 1997, Hamilton [1997] paper was published to add the possibility to transform raw pointer into smart pointer using the Montana C++ programming environment. When the class inherits from `ReferenceCounter`, any raw pointer variable of this class becomes a reference counter:

```

1 #include "ReferenceCounterInterface.h"
2
3 class C : public ReferenceCounter {
```

```
4 public:
5     C(char *name) : ReferenceCounter(name) {}
6 };
7 // ...
8
9 C *cp2 = new C("new C 1");
10 // cp2 is now a smart pointer
```

LISTING 2.3: Montana Smart Pointer

Using this syntax uses less characters (using the same syntax as raw pointer) but it can be confusing for someone that does not know it is a smart pointer under the hood. Also, the paper being old, the tool used (“Montana”) no longer seems to exist.

2.4 Lifetime of a Variable

The subject of the dissertation is about C++, but it is not the only language that implemented automatic memory management. We will now look at another language feature for automatic memory management in the Rust programming language.

In every languages, variables have a scope. It is the time they hold their values, and once out of scope the compiler / interpreter will report an error if we try to access them. Once they go out of scope, they are deleted, garbage collected, etc.

In the Rust programming language, we can add lifetime annotation to variables. These annotations tell the scope of the variable. The borrow checker can then use these annotations to detect more errors. There are different syntax to add lifetime depending on the context. We can see an example of lifetime annotation for a small function:

```
1 fn smaller<'a>(x: &'a i32, y: &'a i32) -> &'a i32 {
2     if x < y {
3         x
4     } else {
5         y
6     }
7 }
```

LISTING 2.4: Rust Example of lifetime annotation

The lifetime annotation of basic variable usage is not necessary, the language will interfere the lifetime.

Clang is on its way to design and implement a lifetime annotation inspired by rust for C++. Here is an example of what the goal is (with the help of some macro definitions to ease the syntax):

```
1 const std::string& $a smaller(const std::string& $a s1, const std::string& $a
    s2) {
2   if (s1 < s2) {
3     return s1;
4   } else {
5     return s2;
6   }
7 }
```

LISTING 2.5: C++ clang lifetime annotation with macros

And there is a paper [de Castro Correia \[2024\]](#) that adds lifetime annotation to C through an external tool step. This tool needs to be installed separately, and needs to be plumbed in the build pipeline.

```
1 #pragma coral lf x = %a
2 #pragma coral lf y = %b
3 #pragma coral lf return = %a
4 int const *return_x(int const *x, int const *y) { return x; }
```

LISTING 2.6: C example of lifetime annotation

Lifetime annotations to variables improve the borrow checker at the cost of a more verbose syntax. Thus, the cost is on the programmer.

2.5 C++ New Syntax Projects Alternative: Cpp2 and cppfront

Herb Sutter, the president of the Standard C++ Foundation, is developing cppfront ([Sutter \[2025\]](#)), a tool to “compile” cpp2 to C++. The name is a reference to how C++ was first created. The goal is not to be a new language, but to integrate with C++. The cpp2 syntax aims to be simpler and safer:

- Types for every construct can be omitted for the cpp2 to try to deduce it.
- Values are always initialized before being used.
- The “**new**” keyword uses a Unique smart pointer.
- All values are “**const**” by default except local variable.
- Parameters of functions and methods have a kind. This specify if it is read-only / write-only / read-write / passed by copy / taking ownership.
- Loops can have a label, break and continue can then specify which loop to break/-continue.
- Pre and Post condition (contracts) for functions and methods.
- “**this**”, the variable representing the current object in a method is not a pointer to this object but a parameter.
- If a methods is not implemented, but C++ creates it by default, cpp2 will generate codes to tell the C++ compiler to not generate this method.
- Subscripts bounds check for vectors by default.
- Checks for comparison between different types by default.
- Runtime checks for division by zero.
- Runtime checks for dereference of the null pointer.

In cpp2, there is two ways to ask the compiler to deduce a variable type when it is declared: use the “**_**” instead of writing the type, or don’t write the type. Because any variable declaration is under the form “**x : type = value**”, when choosing the later and removing the type, the syntax become “**x := value**”. This is different than C++ where, to ask the compiler to deduce the type, we need the keyword “**auto**”: “**auto x = value**”. Using complex type requires less effort from the developer when the type can be omitted as it is the case in cpp2, and C++ but with the added extra cost of writing “**auto**”.

With cpp2, C++ objects can be dynamically allocated with “**unique.new**” or “**shared.new**”. If “**unique**” and “**shared**” is not written before, it default to “**unique.new**”.

```

1 value1 := unique.new<int>(1);
2 value2 := shared.new<int>(2);
3 value3 := new<int>(3);

```

LISTING 2.7: Cpp2 unique and shared

We see that the “**new**” keyword is always a smart pointer in cpp2.

2.6 Evaluating a Syntax

In this dissertation we want to design and implement a new syntax for smart pointers. We need a way to evaluate this syntax. To measure code complexity, the Halstead metrics, and Cyclomatic Complexity (also called “ $V(G)$ ”) are two of the most known metrics.

2.6.1 Cyclomatic Complexity

Cyclomatic Complexity focus on the control flow graph of the program. The nodes represent a sequential code, a program entry, and a program end. The metric “ $V(G)$ ” is then computed using the number of linearly independent paths to each of the nodes (Kafura [2025]). This metric is tightly connected to the number of independent conditional branching the program has. It has no relation to code syntax, therefore it will not be used in this dissertation.

2.6.2 The Halstead Metrics

Haslthead Metrics, in comparison is based on the grammar of the programming language: the number of operand and operator used in the code. Because we want to design a new syntax that does not change the amount of branching, but changes the grammar, we will focus on the Halstead Metrics.

Halstead Metrics uses 4 numbers:

- η_1 = the number of distinct operators.

- η_2 = the number of distinct operands.
- N_1 = the total number of operators.
- N_2 = the total number of operands.

With this numbers we can get:

- Program Vocabulary $\eta = \eta_1 + \eta_2$
- Program Length $N = N_1 + N_2$
- Calculated Estimated Program Length : $\hat{N} = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$
- Volume $V = N * \log_2 \eta_2$
- Difficulty $D = \frac{\eta_1}{2} * \frac{N_2}{\eta_2}$
- Effort $E = D * V$
- Time Required to Program $T = \frac{E}{18}$
- Number of Delivered Bugs $B = \frac{E^{\frac{2}{3}}}{3000}$

To use this metrics we need to define the operators and operands of C++. For C++, [Verysoft \[2025\]](#) suggests that for the operands, we have the:

- identifier.
- typename.
- typespec = `bool`, `char`, `double`, `float`, `int`, `long`, `short`, `signed`, `unsigned`, `void`.
- constant.

And for the operators:

- storage class specifiers = `auto`, `extern`, `inline`, `register`, `static`, `typedef`, `virtual`, `mutable`.
- type qualifier = `const`, `friend`, `volatile`.
- reserved keywords = `asm`, `break`, `case`, `class`, `continue`, `default`, `delete`, `do`, `else`, `enum`, `for`, `goto`, `if`, `new`, `operator`, `private`, `protected`, `public`, `return`, `sizeof`, `struct`, `switch`, `this`, `union`, `while`, `namespace`, `using`, `try`, `catch`, `throw`, `const_cast`, `static_cast`, `dynamic_cast`, `reinterpret_cast`, `typeid`, `template`, `explicit`, `true`, `false`, `typename`.

- operators = `! != % %= & && || &= ( ) * *= + ++ += , - -- -= -> . ... / /= : ::`
`< << <= <= == > >= >> >>= ? [ ] ^ ^= { } | |= ~.`
- some control structures are special cases (the parenthesis and colon [in the switch case] do not count as one more operator) = `for (...) if (...) switch (...) while (...) catch (...)`

But that does not contain the new reserved keywords that C++ added. For the operands, we can add: `char8_t char16_t char32_t`. And for the operators: `alignas alignof and and_eq atomic_cancel atomic_commit atomic_noexcept bitand bitor compl concept consteval constexpr constinit contract_assert co_await co_return co_yield decltype export mutable noexcept not not_eq nullptr or or_eq reflexpr requires static_assert synchronized thread_local virtual wchar_t xor xor_eq`

2.7 Guidelines for a New Syntax

We looked at alternatives programming languages and syntaxes, we also found a way to evaluate the syntax complexity. From this part we can highlight some ideas for our syntax:

- It will not hijack an already existing syntax. We saw with the Montana Reference Counter Pointer that it can be difficult to understand where a smart pointer is used. Hijacking a functionality of the C++ language can break code compatibility. Our goal is to integrate with the language, not creating a new one.
- It needs to be as succinct as possible without being too hard to understand. When looking at lifetime annotation to handle memory management, we saw that it can be hard to understand the code when the syntax get in the way of the code logic.
- It needs to be at least the same level of complexity of the existing syntax for smart pointers. Creating a syntax to help the usage of smart pointer should not introduce more complexity (evaluated with the Halstead Metrics).
- It needs to be tightly integrated to the compiler, to ease the use without requiring an external tool and modify the build pipeline.

Chapter 3

Specifications

3.1 Syntax Design Specifications

3.1.1 SDS1

The syntax must be compatible with the C++ standard.

Existing C++ code should not break. Breaking code can impact adoption of a new syntax.

3.1.2 SDS2

The syntax could be customizable.

The developer will have the possibility to change the behaviour of the smart pointer and the syntax used.

3.1.3 SDS3

The syntax should be ergonomic.

Every developer should have the possibility to code with this syntax. Basic characters on the vast majority of keyboards should be used.

3.1.4 SDS4

The feature must be easy to understand in a documentation.

We want to ease the use of smart pointer, therefore, the feature should be easy to understand.

3.2 Technical Specifications

3.2.1 TS1

The modification should not increase the compilation time by more than 0.1 second for a file.

It is easier to work incrementally with a tool that is fast.

3.2.2 TS2

The output program using the syntax should not have more performance overhead than using the current syntax.

Implementing a sugar syntax should not impact runtime performance.

3.2.3 TS3

The modified parts could be tightly close in the Clang code.

The more there is modification in different context, the more difficult it is to explain and implement again.

3.2.4 TS4

LLVM tools could comprehend the new syntax.

Modifying Clang, which is integrated in the LLVM project, could help other LLVM tools to parse correctly the syntax.

Chapter 4

Implementation

This part describes the implementation of the new syntax. The Clang repository is forked with the `8fddef8483dc9eb569580ffd13695b8f54d3c058` git hash.

We will first explain the clang parts that are related to our work. We will then have an overview of the changes to the code. Finally, we will present some alternative approaches that have been envisaged.

4.1 Understanding the Clang Compiler Internals

Core components of Clang

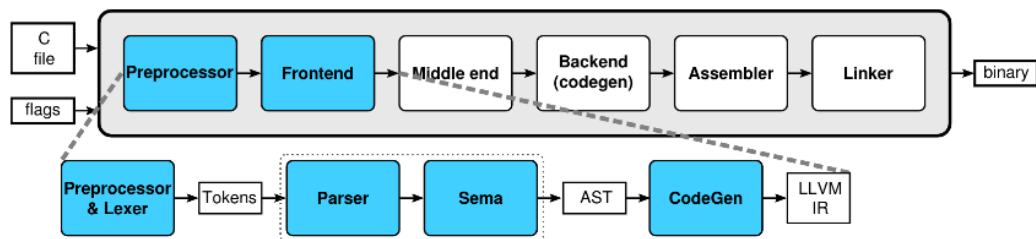


FIGURE 4.1: Understanding the core components of the Clang compiler driver, [Slide from van Haastregt and Stulova \[2019\]](#)

The Clang Compiler is separated into multiple parts. These parts are ([van Haastregt and Stulova \[2019\]](#)):

- Preprocessor and Lexer. Outputs tokens.
- Parser. Feeds the Semantic Analyser.
- Semantic Analyser (Sema). Outputs AST.
- CodeGen. Outputs LLVM Intermediate Representation (IR).

4.1.1 Overview of the Clang Parser

Because clang can parse C and C++ code, but C++ code has a lot of C syntax, the parser has parts that parse specific C++ syntax in the same file. The distinction of the language being parsed is inside a class called “LangOptions”. This class can be accessed within the Parser with its method “getLangOpts()”. It contains various informations: the language being parsed, but also the standard used (i.e.: c++11 or c++17 or ...) and some other platform extensions.

The parser is a recursive decent parser. This means it will try to parse large group of code first (i.e.: class, function, scopes) and then reduce to little blocks (i.e.: declarations, assignment, expressions). The benefit for us is that changing the behaviour of the compiler when parsing a type will change it for every types parsing.

Each time the parser has validated something, it calls the Sema (Semantic Analyser) to process the piece of data. The Sema then proceeds to some validation steps and stores the data in some lookup table and constructs an AST node.

4.1.2 How Templates are processed in the Clang Parser

In this part, all method calls without a scope specifier are in the Parser class. A figure is available to summarize the call’s logic: [4.2](#).

Templates constructions are not widely documented in the Clang CFE Internals ([Clang \[2025\]](#)), so we have to trace the functions call’s stack with a C++ code using templates.

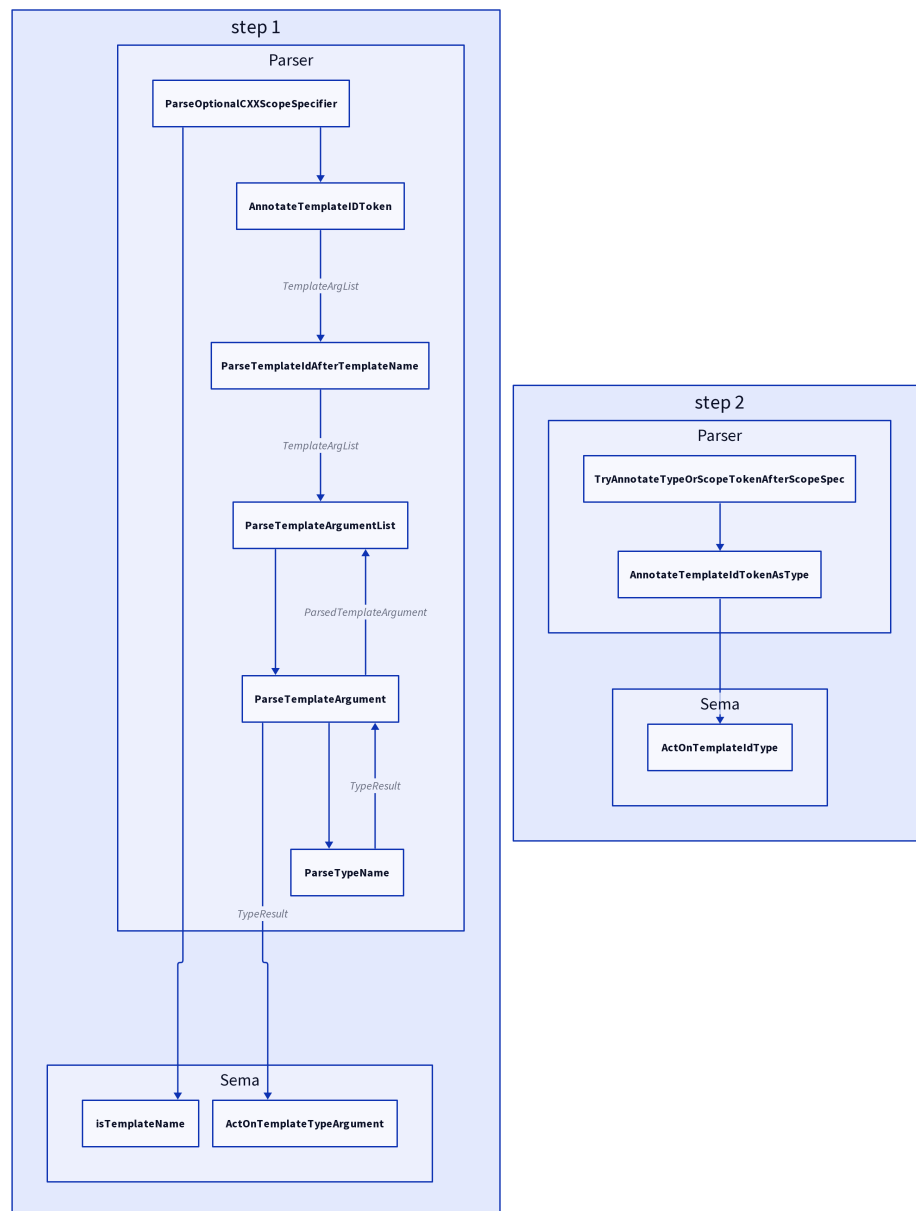


FIGURE 4.2: Template Calls

Templates starts being parsed in the `ParseOptionalCXXScopeSpecifier` method. This method will take the scope namespace if there is some, then if it sees a “less” (<) will understand the identifier of the template class name.

It will then proceed to check if the identifier corresponds to a defined template class name with the call to the `Sema::isTemplateName` method.

`ParseOptionalCXXScopeSpecifier` will then call `AnnotateTemplateIdToken`. This

method proceeds to parse the full template tokens (i.e. `identifier < ... >`) and injects back a unique annotation token.

`AnnotateTemplateIdToken` first calls `ParseTemplateIdAfterTemplateName` that calls `ParseTemplateArgumentList` to parse all the things between the “less” (`<`) and “greater” (`>`). This method calls `ParseTemplateArgument` until `TryConsumeToken(tok::comma)` returns false. All the parsed template arguments are added to a vector that given as a function parameter that was created in `AnnotateTemplateIdToken`.

`ParseTemplateArgument`, in the case of a type, will call `ParseTypeName`. The result is then sent to `Sema::ActOnTemplateTypeArgument`.

Then, in `AnnotateTemplateIdToken`, an annotation token value is created with the vector of template argument, the template class name informations. The current token is modified to be a `tok::annot_template_id`, and its annotation value is set to the value created.

We then exit the `ParseOptionalCXXScopeSpecifier`. Later, in the `TryAnnotateTypeOrScopeTokenAfterScopeSpec`, it detects that the token is a `tok::annot_template_id` and calls `AnnotateTemplateIdTokenAsType`.

`AnnotateTemplateIdTokenAsType` calls `Sema::ActOnTemplateIdType` that gives back a `TypeResult` (This is the same type that is returned by `ParseTypeName`). The current token is then being modified to a `tok::annot_typename`, and its value is set to the `TypeResult` value.

Later, in a different context: `ParseDeclarationSpecifiers`, it will see the `tok::annot_typename` and use it as the type.

4.1.3 Where a Raw Pointers Gets Parsed

In some case, C and C++ can specify multiple variables on the same line, using the same type. Some type specifier only applies to one variable being declared, and other applies to all the variable.

The “pointer” type applies only to one variable. In the “`Parser::ParseDeclaratorInternal`”, the function “`isPtrOperatorToken`” is called to check if a token might be a “pointer”

(reference, pointer). In the case it is not, “`Parser::ParseDeclaratorInternal`” returns early. If it is, the type of pointer is added as a “`DeclaratorChunk`”.

A “`DeclaratorChunk`” is a piece of data added to a “`Declarator`”. A “`Declarator`” is one of the variable, and a “`DeclarationSpecifier`” applies to all the variable being declared.

There is also one other functions that check if it is a pointers: “`Parser::MaybeDeclarator`”.

4.1.4 The Clang Driver Command Line Flags

This part is largely documented in the Clang Internal Manual ([Clang \[2025\]](#)).

To add a new flag, we need to create the variable that will be holding the value. It can be put in any of the class available depending of the flag context. For example, if the flag is modify how a language is interpreted, it can be in the “`LangOptions`” class (“clang/include/clang/Basic/LangOptions.h”).

Then we need to modify the “clang/include/clang/Driver/Options.td”. A number of helper syntax can be used to help parsing common flag syntax easily. We need to specify the name, the flag “visibility” (as in which part the flag is used, i.e.: preprocessor/compilation/linker/...), the flag type (single/double dash, with equals or space delimiter if an argument is given), a help text, the data type being passed as an argument if any (i.e.: string, string vector with argument separated by space, number, and other types).

If the option can’t be parsed by the provided helpers function, one need to code the parsing function. Otherwise, a two liner statement can be added in the clang driver command line parser.

Finally, we can use the attribute of the class.

4.2 Modifying Clang

4.2.1 Adding a New Clang Command Line Flag

The dissertation aims to modify the C++ language syntax, so the choice of class for the command line variable is in the `LangOptions` class (“clang/include/clang/Basic/LangOptions.h”).

The syntax is a dash, followed by the `smart-pointer=`. For the “visibility”, our syntax is applied during the compiler phase, and its a Clang option.

Because it will be possible to specify the flag multiple times, we parse a string vector.

The command line syntax is particular: a vector of each argument given to the command line instance of the the flag. And withing each instance of the flag, its value is separated by a comma. A method is added to get a smart pointer class representation (string type) from a token. The function used to get the character string from the token is “`getPunctuatorSpelling`”. It works only for “punctuator” (i.e.: “!” or %, etc.)

See [4.3](#) for an overview of how it looks like.

4.2.2 Adding a New DeclaratorChunk

The class is in the “clang/include/clang/Sema/DeclSpec.h” file. We add a `SmartPointer` enum “`Kind`”, And an inner struct containing the token that flagged the smart pointer syntax in the code. This will be used to distinguish which type of smart pointer to use.

We then modify the “`destroy`” method to call the `SmartPointer` inner struct if the “`Kind`” is set to a `SmartPointer`.

And finally, we create a static method to create a new “`DeclaratorChunk`” containing a `SmartPointer` inner struct.

4.2.3 Accept the New Syntax

After the previous modifications, we can edit the `Parser` (“clang/lib/Parse/ParseDecl.cpp”) to accept a smart pointer syntax. We modify the methods mentioned earlier 4.1.3 to accept a new token where initially only raw pointer and reference were accepted.

When a smart pointer syntax is detected, we add a `DeclaratorChunk` to the `Declarator` by calling the method we created in 4.2.2.

4.2.4 Modify the Type

At some point in the `Sema` class, the function `GetFullTypeForDeclarator` (“clang/lib/Sema/SemaType.cpp”) is called. This function modifies the type with the data available in the “`DeclaratorChunks`”.

First, we get the smart pointer template class given by the command line flag, and with the token parsed before as a key.

To create a template type, we first need to create the scope (`CXXScopeSpec`) in which the template resides. This is done by searching for the namespace name, and extending the scope with it.

Then, we need to get the template name (`OpaquePtr<TemplateName>`) by using the template name string as an identifier and checking if it is valid in the scope. The method responsible to check the validity sets the value of our template name.

We now have our template type, but we need to add the typename specialisation. We create a `TemplateArgList`, and append the current initial type of the variable to it. After creating this, we can now clear the initial type of the declaration specifier, and set its value with the one we created.

4.3 Final Implementation

The final design of the syntax is:

```
1 T% my_var;
```

LISTING 4.1: Improved Syntax Design

It is the same syntax as a raw pointer, except the character (token) used is the “%” (percent) instead of the “*” (star). The use of the percent token is not fixed. We will provide a command line to let users specify the token and the smart pointer used.

In the standard, there is 3 types of smart pointer: “unique”, “shared”, “weak”. To allow customisation of the syntax, a command line flag will be provided. It will allow multiple mention of the flag to add more than one smart pointer usage at the same time.

```
1 clang++ \  
2     -smart-pointer=%,std::shared_ptr  \  
3     '-smart-pointer=|,std::unique_ptr' \  
4     a.c  
5 clang++  -smart-pointer=%,std::weak_ptr      a.c  
6 clang++  '-smart-pointer=|,std::unique_ptr'  a.c
```

LISTING 4.2: Clang Flags to Change Smart Pointer

The smart pointer class can be any class. It is not required that it is one provided by the standard library.

4.4 Alternative Approaches

4.4.1 Why Not Changing the Tokens in the Lexer ?

At first, the author thought that changing the tokens directly during the lexer phase would be easier. It is easier to add/modify/remove tokens, but we need to know when the type parsed will end.

```
1 int my_val;  
2  
3 int *my_val1;  
4  
5 const int my_val2;
```

LISTING 4.3: Type Examples

The Lexer don't know that “int” is a type. So we could put a token first that declare the next token as the type to our smart pointer. But then, how can we know when the type stops ? The “*” is part of the “int *” type.

4.4.2 Why Not Changing the AST Type during Code Generation ?

The Clang AST part is another pass in the driver. The less each pass are modified, the less there is potential bug. Also, Clang ([Clang \[2025\]](#)) declare that when the AST is generated, it is as much static as possible. Modifying the AST after it has been generated is not recommended.

There is some methods in the `Sema` class to rebuild a type after it has been created, but we didn't have to implement them as we succeeded to implement the smart pointer syntax. Also, at first glance, the method seems more complex to use than modifying the type when it is created.

4.4.3 Why Using a Command Line Flag instead of a pragma directive ?

When reading the code responsible for the `pragama` directive, one thing invalidated this possibility: the “`LangOptions`” class is a constant in this scope. This means we could not change our smart pointer behaviour from a `Pragma`. `Pragama` can only attach some values to a statement. This means, we would have to add something else to the statements information to hold our Smart Pointer syntax, and then retrieve it in some way in the Parser. This seems overcomplicated, and the command line argument was easier to comprehend.

Chapter 5

Evaluation

5.1 Design Evaluation - Halstead Metrics

Using the Halstead Metrics [2.6.2](#), we will now evaluate how the syntax for smart pointers suggested in this dissertation compare to the current one.

In this section, the numbers in the tables [5.1](#), [5.2](#), [5.3](#), [5.4](#), [5.5](#) and [5.6](#) will be truncated to two decimal point, except when needed for comparison when two decimal point is not enough.

5.1.1 Example 1

We will start with a basic example.

```
1 #include <memory>
2 // new syntax
3 int main() {
4     int %value = std::make_unique<int>(1);
5 }
6 // current syntax
7 int main() {
8     std::unique_ptr<int> value = std::make_unique<int>(1);
9 }
```

LISTING 5.1: Halstead Evaluation

For the new syntax, we have,

N_1	Total Number of Operators	11
η_1	Number of Distinct Operators	9
N_2	Total Number of Operand	7
η_2	Number of Distinct Operands	5
η	Program Vocabulary	$\eta_1 + \eta_2 = \mathbf{14}$
N	Program Length	$N_1 + N_2 = \mathbf{18}$
$\hat{N}$	Calculated Estimated Program Length	$\eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2 = \mathbf{40.13}$
V	Volume	$N * \log_2 \eta_2 = \mathbf{41.79}$
D	Difficulty	$\frac{\eta_1}{2} * \frac{N_2}{\eta_2} = \mathbf{6.3}$
E	Effort	$D * V = \mathbf{263.30}$
T	Time Required to Program	$\frac{E}{18} = \mathbf{14.62}$
B	Number of Delivered Bugs	$\frac{E^3}{3000} = \mathbf{0.013}$

TABLE 5.1: Halstead Metrics for 5.1, New Syntax

N_1	Total Number of Operators	11
η_1	Number of Distinct Operators	8
N_2	Total Number of Operand	9
η_2	Number of Distinct Operands	6
η	Program Vocabulary	$\eta_1 + \eta_2 = \mathbf{14}$
N	Program Length	$N_1 + N_2 = \mathbf{20}$
$\hat{N}$	Calculated Estimated Program Length	$\eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2 = \mathbf{39.50}$
V	Volume	$N * \log_2 \eta_2 = \mathbf{51.69}$
D	Difficulty	$\frac{\eta_1}{2} * \frac{N_2}{\eta_2} = \mathbf{6}$
E	Effort	$D * V = \mathbf{310.19}$
T	Time Required to Program	$\frac{E}{18} = \mathbf{17.23}$
B	Number of Delivered Bugs	$\frac{E^3}{3000} = \mathbf{0.015}$

TABLE 5.2: Halstead Metrics for 5.1, Current Syntax

- Operators: `( ) { % = :: < > ( ) }`
- Operators (Distinct): `( ) { % = :: < > }`
- Operands: `int main int value std make_unique int`
- Operands (Distinct): `int main value std make_unique`

and for the current syntax,

- Operators: `( ) { :: = :: < > ( ) }`
- Operators (Distinct): `( ) { :: = < > }`
- Operands: `int main std unique_ptr int value std make_unique int`

- Operands (Distinct): `int main std unique_ptr value make_unique`

From the tables 5.1 and 5.2, we can see that even with a longer program length, the current syntax is less difficult. But the effort to program with the current syntax is higher than the syntax designed in this dissertation.

We could argue that in this case, we could replace the type with the `auto` keyword. In the next example, the use of keyword is not possible.

5.1.2 Example 2

```

1 #include <memory>
2 // new syntax
3 int some_func(int %value) { return *value; }
4 // current syntax
5 int some_func(std::unique_ptr<int> value) { return *value; }

```

LISTING 5.2: Haslstead Evaluation

N_1	Total Number of Operators	7
η_1	Number of Distinct Operators	7
N_2	Total Number of Operand	5
η_2	Number of Distinct Operands	3
η	Program Vocabulary	$\eta_1 + \eta_2 = 10$
N	Program Length	$N_1 + N_2 = 12$
$\hat{N}$	Calculated Estimated Program Length	$\eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2 = 24.40$
V	Volume	$N * \log_2 \eta_2 = 19.01$
D	Difficulty	$\frac{\eta_1}{2} * \frac{N_2}{\eta_2} = 5.83$
E	Effort	$D * V = 110.94$
T	Time Required to Program	$\frac{E}{18} = 6.16$
B	Number of Delivered Bugs	$\frac{E^2}{3000} = 0.007$

TABLE 5.3: Halstead Metrics for 5.2, New Syntax

For the new syntax, we have,

- Operators: `( % ) { return * }`
- Operators (Distinct): `( % ) { return * }`
- Operands: `int some_func int value value`
- Operands (Distinct): `int some_func value`

N_1	Total Number of Operators	9
η_1	Number of Distinct Operators	9
N_2	Total Number of Operand	7
η_2	Number of Distinct Operands	5
η	Program Vocabulary	$\eta_1 + \eta_2 = \mathbf{14}$
N	Program Length	$N_1 + N_2 = \mathbf{16}$
$\hat{N}$	Calculated Estimated Program Length	$\eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2 = \mathbf{40.13}$
V	Volume	$N * \log_2 \eta_2 = \mathbf{37.15}$
D	Difficulty	$\frac{\eta_1}{2} * \frac{N_2}{\eta_2} = \mathbf{6.3}$
E	Effort	$D * V = \mathbf{234.05}$
T	Time Required to Program	$\frac{E}{18} = \mathbf{13.00}$
B	Number of Delivered Bugs	$\frac{E^{\frac{2}{3}}}{3000} = \mathbf{0.012}$

TABLE 5.4: Halstead Metrics for 5.2, Current Syntax

and for the current syntax,

- Operators: (:: < >) { return * }
- Operators (Distinct): (:: < >) { return * }
- Operands: int some_func std unique_ptr int value value
- Operands (Distinct): int some_func std unique_ptr value

In this example, the results of 5.3 and 5.4 show that the new syntax is less difficult, and requires less effort.

5.1.3 Example 3

We tested simple case for our algorithm. We will now use the quick sort algorithm to test the syntax with the Halstead Metrics. This functions takes a shared pointer as an argument to modify its value in place.

```

1 // new syntax
2 int partition(std::vector<int> %vec, int low, int high)
3 // current syntax
4 int partition(std::shared_ptr<std::vector<int>> vec, int low, int high)
5
6 // new syntax
7 void quick_sort(std::vector<int> %vec, int low, int high)
8 // current syntax

```

```

9 void quick_sort(std::shared_ptr<std::vector<int>> vec, int low, int high)
10 // end

```

LISTING 5.3: Haslthead Evaluation (Quick Sort algorithm)

N_1	Total Number of Operators	16
η_1	Number of Distinct Operators	7
N_2	Total Number of Operand	20
η_2	Number of Distinct Operands	9
η	Program Vocabulary	$\eta_1 + \eta_2 = \mathbf{16}$
N	Program Length	$N_1 + N_2 = \mathbf{36}$
$\hat{N}$	Calculated Estimated Program Length	$\eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2 = \mathbf{48.18}$
V	Volume	$N * \log_2 \eta_2 = \mathbf{114.11}$
D	Difficulty	$\frac{\eta_1}{2} * \frac{N_2}{\eta_2} = \mathbf{7.77}$
E	Effort	$D * V = \mathbf{887.57}$
T	Time Required to Program	$\frac{E}{18} = \mathbf{49.30}$
B	Number of Delivered Bugs	$\frac{E^{\frac{2}{3}}}{3000} = \mathbf{0.0307}$

TABLE 5.5: Halstead Metrics for 5.3, New Syntax

N_1	Total Number of Operators	20
η_1	Number of Distinct Operators	6
N_2	Total Number of Operand	24
η_2	Number of Distinct Operands	10
η	Program Vocabulary	$\eta_1 + \eta_2 = \mathbf{16}$
N	Program Length	$N_1 + N_2 = \mathbf{44}$
$\hat{N}$	Calculated Estimated Program Length	$\eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2 = \mathbf{48.72}$
V	Volume	$N * \log_2 \eta_2 = \mathbf{146.16}$
D	Difficulty	$\frac{\eta_1}{2} * \frac{N_2}{\eta_2} = \mathbf{7.19}$
E	Effort	$D * V = \mathbf{1052.38}$
T	Time Required to Program	$\frac{E}{18} = \mathbf{58.46}$
B	Number of Delivered Bugs	$\frac{E^{\frac{2}{3}}}{3000} = \mathbf{0.0344}$

TABLE 5.6: Halstead Metrics for 5.3, Current Syntax

For readability, only the changed lines are displayed. The full code is available in the Appendices (B)

For the new syntax, we have,

- Operators: (:: < > % , ,) (:: < > % , ,)
- Operators (Distinct): (:: < > % , ,)
- Operands: int partition std vector int vec int low int high void quick_sort
std vector int vec int low int high

- Operands (Distinct): `int partition std vector vec low high void quick_sort`

and for the current syntax,

- Operators: `( :: < :: < > , , ) ( :: < :: < > , , )`
- Operators (Distinct): `( :: < > , )`
- Operands: `int partition std shared_ptr std vector int vec int low int high void quick_sort std shared_ptr std vector int vec int low int high`
- Operands (Distinct): `int partition std shared_ptr vector vec low high void quick_sort`

We can see from the tables 5.5 and 5.6 that the new syntax is more difficult, but requires less effort to use. Although we could argue that the current syntax is more difficult to read with a template inside a template: the full type is 33 characters wide versus 18 for the new syntax.

5.1.4 Summary

From the different examples, we can see that the syntax designed in this dissertation can be harder (in difficulty) sometimes, but it requires always less effort. We answered our goal to design a syntax that will be less cumbersome and easier to write (less effort) although it can be harder to understand according to the Halstead Metrics.

5.2 Technical evaluation

To benchmark multiple program, we will use “hyperfine”. It is a program that run the different command we provide multiples times and output statistics about the time it took.

The benchmark were done on a consumer laptop running Linux, with 16GB of RAM, and an Intel i7-12700H CPU.

The benchmarks scripts are provided in the Appendices.

5.2.1 Performance of the Compiler

To test the performance of the compiler, we will be compiling the qBittorrent project. This tool is a bittorrent client, programmed in C++, and it uses smart pointer.

We will compile multiples times the project with and without our syntax with our modified clang. We will also compile the project with an unmodified version of clang. For reproducibility, the script used to compute this benchmark is available in the Appendices (C)

```

1 Benchmark 1: qbittorrent_unmodified_clang_modified
2   Time (mean  $\pm \sigma$ ):      103.405 s  $\pm$  0.226 s      [User: 844.579 s, System:
   44.587 s]
3   Range (min ... max):      103.117 s ... 103.749 s      10 runs
4
5 Benchmark 2: qbittorrent_unmodified_clang_unmodified
6   Time (mean  $\pm \sigma$ ):      103.806 s  $\pm$  1.145 s      [User: 848.076 s, System:
   44.894 s]
7   Range (min ... max):      102.686 s ... 105.818 s      10 runs
8
9 Benchmark 3: qbittorrent_modified_clang_modified
10  Time (mean  $\pm \sigma$ ):      106.170 s  $\pm$  5.799 s      [User: 866.825 s, System:
   45.192 s]
11  Range (min ... max):      103.249 s ... 121.148 s      10 runs
12
13 Summary
14  qbittorrent_unmodified_clang_modified ran
15    1.00  $\pm$  0.01 times faster than qbittorrent_unmodified_clang_unmodified
16    1.03  $\pm$  0.06 times faster than qbittorrent_modified_clang_modified

```

LISTING 5.4: Compiler Benchmark

We can see that the modified and unmodified clang compiler takes about the same time to run on the unmodified source of qBittorrent. When qBittorrent use the syntax designed in this dissertation with the modified clang compiler, it takes about 103% to 106% of the time to run. This means 3 seconds more on average.

5.2.2 Performance of the Output Binary

We will test the sorting algorithm, with our syntax and with the current syntax using the modified Clang compiler.

```

1 Benchmark 1: new syntax - 1 2 3 4 5 6 7 8 9 10
2   Time (mean  $\pm \sigma$ ):      1.1 ms  $\pm$  0.2 ms    [User: 0.5 ms, System: 0.5 ms]
3   Range (min ... max):      0.7 ms ... 2.4 ms    10000 runs
4
5 Benchmark 2: current syntax - 1 2 3 4 5 6 7 8 9 10
6   Time (mean  $\pm \sigma$ ):      1.1 ms  $\pm$  0.2 ms    [User: 0.5 ms, System: 0.5 ms]
7   Range (min ... max):      0.7 ms ... 3.0 ms    10000 runs
8
9 Summary
10  current syntax - 1 2 3 4 5 6 7 8 9 10 ran
11    1.00  $\pm$  0.21 times faster than new syntax - 1 2 3 4 5 6 7 8 9 10
12
13 ---
14
15 Benchmark 1: new syntax - random
16   Time (mean  $\pm \sigma$ ):      4.0 ms  $\pm$  0.5 ms    [User: 1.7 ms, System: 2.1 ms]
17   Range (min ... max):      2.5 ms ... 6.0 ms    5000 runs
18
19 Benchmark 2: current syntax - random
20   Time (mean  $\pm \sigma$ ):      4.0 ms  $\pm$  0.5 ms    [User: 1.7 ms, System: 2.1 ms]
21   Range (min ... max):      2.4 ms ... 6.1 ms    5000 runs
22
23 Summary
24  new syntax - random ran
25    1.00  $\pm$  0.17 times faster than current syntax - random

```

LISTING 5.5: Output Binary Benchmark

We can see from the benchmark results that using the new syntax does not impact the performance of the output binary.

5.3 Clang Tests

The Clang compiler has a test suite that can be run. It is available using the `make check-clang`. We can then compare the results with and without our modifications.

The results for the unmodified Clang source code is:

```
1 Total Discovered Tests: 71224
2   Skipped           :    15 (0.02%)
3   Unsupported       :   4052 (5.69%)
4   Passed            :  67130 (94.25%)
5   Expectedly Failed:    27 (0.04%)
6 [100%] Built target check-clang
```

LISTING 5.6: Unmodified Clang Tests

The result for the modified Clang source code is:

```
1 Total Discovered Tests: 46668
2   Skipped           :    10 (0.02%)
3   Unsupported       :   4052 (8.68%)
4   Passed            :  42579 (91.24%)
5   Expectedly Failed:    27 (0.06%)
6 [100%] Built target check-clang
```

LISTING 5.7: Modified Clang Tests

We can confirm that there is not failure when running the tests. The syntax does not break the current syntax.

Chapter 6

Conclusion

In this dissertation, we have designed new syntax for smart pointer that requires less effort to use than the current syntax. We also have modified a C++ compiler (Clang) to compile the syntax and be available for developers to use.

It provides an easier syntax for smart pointer than the current one.

6.1 Summary of the Final Requirements

From the Specifications section (3), we can summarize the results.

6.1.1 Syntax Design Specifications

6.1.1.1 SDS1

The syntax must be compatible with the C++ standard. (3.1.1).

From the Clang unit tests we have run in 5.3, no failure had been reported. This means that existing C++ code can run with our modified Clang.

Because we can let the user choose which token to use for the smart pointer syntax, code can break depending on the choice.

The primary failures plausible by a user choice is because of the trigraphs and digraphs preprocessing. Trigraphs and digraphs are an early feature of C, that was kept in C++.

When developing software was on computer with early keyboard, not all tokens were available on all of them. So other alternative were provided ([CppReference \[2024\]](#)). This preprocessor step is before our parsing of the tokens, so we can't remediate this except disabling trigraphs and digraphs.

6.1.1.2 SDS2

The syntax could be customizable. ([3.1.2](#)).

We provided a way using the Clang command line for the user to choice the token used for the smart pointer, and the class to use.

6.1.1.3 SDS3

The syntax should be ergonomic. ([3.1.3](#)).

The user chooses the token. Depending on the token, the ergonomomy to enter the character can be impacted.

Regarding a general view, the Halstead Metrics tells us that the syntax requires less effort to write.

6.1.1.4 SDS4

The feature should be easy to understand in a documentation ([3.1.4](#)).

The purpose of the feature is to add a sugar syntax for a feature that is already existing. We only need to show the current syntax, and its alternative with the new syntax.

6.1.2 Technical Specifications

6.1.2.1 TS1

The modification should not increase the compilation time by more than 0.1 second for a file. ([3.2.1](#)).

We benchmarked the compilation speed on a C++ project named qBittorrent (5.2.1). The project contains 450 C++ detected files (qBittorrent [2006]). And we noted that there were 3 more seconds on average. Our modified Clang is on track with this technical specifications.

6.1.2.2 TS2

The output program using the syntax should not have more performance overhead than using the current syntax. (3.2.2).

From our benchmark (5.2.2), we have no overhead on the binary.

6.1.2.3 TS3

The modified parts could be tightly close in the Clang code. (3.2.3).

We modified:

- clang/include/clang/Basic/LangOptions.h
- clang/include/clang/Driver/Options.td
- clang/include/clang/Sema/DeclSpec.h
- clang/lib/Driver/ToolChains/Clang.cpp
- clang/lib/Parse/ParseDecl.cpp
- clang/lib/Parse/ParseDeclCXX.cpp
- clang/lib/Sema/DeclSpec.cpp
- clang/lib/Sema/SemaDeclCXX.cpp
- clang/lib/Sema/SemaType.cpp

We only modified the files related to Clang. In this Clang folder, only a little number of file has been modified.

6.1.2.4 TS4

LLVM tools could comprehend the new syntax. (3.2.4).

When compiling the llvm-project and enabling the clang extra tools, we get access to clangd, a language server. A language server enable code completion for code editors. The compiled clangd with our modified Clang can understand the type of a variable with our syntax.

Clangd is an example, but if clangd understand our syntax, other tools will use the same routines that clangd uses, thus, we can say that other clang extra tools are integrated.

6.2 Limits and Future Works

The current implementation works only with smart pointers class that are in only one namespace deep. It does not work for classes in nested namespace, and in classes nested in another classes. We could make this works by integrating these options.

“`std::unique_ptr`” has two different constructor: one with only the underlying type, and one with the underlying type and a “deleter” class. Our sugar syntax only provide the option for the one argument only constructor. We could improve or re design the syntax to make it works for the other constructor.

The implementation being only made on a Clang compiler code fork, it is only available for users compiling themselves the code. The fork will not be updated from the upstream changes. Also, the users wanting to use the syntax will need to switch to Clang, if they previously used the GCC compiler.

Clang can compile C and C++, and the parsing of the two languages are tight. We could use what we have now understand of the Clang compiler to implement the same syntax with the C language. It is possible as GCC has attributes for cleanup (Snaipe [2015]).

Because of the organisation of the C++ language, the only way to suggest this syntax is to submit a proposal. This is not the goal of the dissertation, but a proposal could be made to discuss syntax’s change to smart pointer officially.

6.3 In Closing

The syntax we designed improve efficiency. It can be used on real world project with the public fork available [on the github repository](#).

Dissertation written by

Xavier MITAULT

Supervised by Dr. Adam SAMPSON

Appendix A

External Links

- Clang source code: <https://github.com/llvm/llvm-project>
- Clang modified source code: <https://github.com/Saverio976/llvm-improved-syntax-for-cxx-smart-pointer>
- qBittorrent source code: <https://github.com/qbittorrent/qBittorrent>
- qBittorrent modified source code: <https://github.com/Saverio976/qBittorrent-llvm-improved-syntax-for-cxx-smart-pointer>

Appendix B

Quick Sort Code

B.1 Current Syntax

```
1 #include <memory>
2 #include <vector>
3
4 int partition(std::shared_ptr<std::vector<int>> vec, int low, int high)
5 {
6     int pivot = vec->at(high);
7     int i = low - 1;
8
9     for (int j = low; j <= high - 1; j++) {
10         if (vec->at(j) <= pivot) {
11             i++;
12             std::swap(vec->at(i), vec->at(j));
13         }
14     }
15     std::swap(vec->at(i + 1), vec->at(high));
16     return i + 1;
17 }
18
19 void quick_sort(std::shared_ptr<std::vector<int>> vec, int low, int high)
20 {
21     if (low < high) {
22         int index = partition(vec, low, high);
23         quick_sort(vec, low, index - 1);
24         quick_sort(vec, index + 1, high);
25     }
26 }
```

LISTING B.1: Haslthead Evaluation (Quick Sort algorithm)

B.2 New Syntax

```
1 #include <memory>
2 #include <vector>
3
4 int partition(std::vector<int> %vec, int low, int high)
5 {
6     int pivot = vec->at(high);
7     int i = low - 1;
8
9     for (int j = low; j <= high - 1; j++) {
10         if (vec->at(j) <= pivot) {
11             i++;
12             std::swap(vec->at(i), vec->at(j));
13         }
14     }
15     std::swap(vec->at(i + 1), vec->at(high));
16     return i + 1;
17 }
18
19 void quick_sort(std::vector<int> %vec, int low, int high)
20 {
21     if (low < high) {
22         int index = partition(vec, low, high);
23         quick_sort(vec, low, index - 1);
24         quick_sort(vec, index + 1, high);
25     }
26 }
```

LISTING B.2: Haslthead Evaluation (Quick Sort algotihm)

Appendix C

Script Performance of the Compiler

C.1 Script Performance of the Compiler

```
1 #!/bin/sh
2
3 qbittorrent_url='https://github.com/Saverio976/qBittorrent-llvm-improved-syntax-for-
    cxx-smart-pointer.git'
4 llvmproject_modified='https://github.com/Saverio976/llvm-improved-syntax-for-cxx-smart
    -pointer.git'
5 llvmproject_unmodified='https://github.com/llvm/llvm-project.git'
6
7 git clone "$qbittorrent_url" 'qbittorrent_unmodified_clang_unmodified'
8 git clone "$qbittorrent_url" 'qbittorrent_unmodified_clang_modified'
9 git clone "$qbittorrent_url" 'qbittorrent_modified_clang_modified'
10 git clone "$llvmproject_unmodified" 'llvm-project-unmodified'
11 git clone "$llvmproject_modified" 'llvm-project-modified'
12
13 # llvm-project-modified
14 (
15     cur_path="$(pwd) "
16     cd llvm-project-modified
17     git checkout 'feat/improved_smart_pointer_syntax'
18     mkdir -p build
19     cd build
20     cmake "-DCMAKE_INSTALL_PREFIX=$cur_path/install" "-DLLVM_ENABLE_PROJECTS=clang" "-
        DLLVM_TARGETS_TO_BUILD=X86" "-DCMAKE_BUILD_TYPE=Release" "../llvm"
21     make
22 )
```

```

23 # llvm-project-unmodified
24 (
25     cur_path="$(pwd) "
26     cd llvm-project-unmodified
27     git checkout '8fddef8483dc9eb569580ffd13695b8f54d3c058'
28     mkdir -p build
29     cd build
30     cmake "-DCMAKE_INSTALL_PREFIX=$cur_path/install" "-DLLVM_ENABLE_PROJECTS=clang" "-
        DLLVM_TARGETS_TO_BUILD=X86" "-DCMAKE_BUILD_TYPE=Release" "../llvm"
31     make
32 )
33
34 # 1
35 # setup qbittorrent_unmodified.clang_unmodified
36 test1_name='qbittorrent_unmodified.clang_unmodified'
37 test1_conclude="rm -rf $test1_name/build"
38 test1_prepare="cmake -B $test1_name/build -S $test1_name -DCMAKE_CXX_COMPILER=\"$(pwd)
        /llvm-project-unmodified/build/bin/clang++\""
39 test1_command="make -C $test1_name/build -j8"
40 (
41     cd qbittorrent_unmodified.clang_unmodified
42     git checkout 'unmodified'
43 )
44
45 # 2
46 # setup qbittorrent_unmodified.clang_modified
47 test2_name='qbittorrent_unmodified.clang_modified'
48 test2_conclude="rm -rf $test2_name/build"
49 test2_prepare="cmake -B $test2_name/build -S $test2_name -DCMAKE_CXX_COMPILER=\"$(pwd)
        /llvm-project-modified/build/bin/clang++\""
50 test2_command="make -C $test2_name/build -j8"
51 (
52     cd qbittorrent_unmodified.clang_modified
53     git checkout 'unmodified'
54 )
55
56 # 3
57 # setup qbittorrent_modified.clang_modified
58 test3_name='qbittorrent_modified.clang_modified'
59 test3_conclude="rm -rf $test3_name/build"
60 test3_prepare="cmake -B $test3_name/build -S $test3_name \
        -DCMAKE_CXX_COMPILER=$(pwd)
        /llvm-project-modified/build/bin/clang++" \
        -DCMAKE_CXX_FLAGS=-smart-pointer=%,
        std::unique_ptr -smart-pointer='|,std::shared_ptr'"
61 test3_command="make -C $test3_name/build -j8"
62 (
63     cd qbittorrent_modified.clang_modified
64     git checkout 'modified'
65 )

```

```
66
67 hyperfine \
68     --shell none \
69     --warmup 1 \
70     --min-runs 10 \
71     --command-name "$test2_name" \
72     --conclude "$test2_conclude" \
73     --prepare "$test2_prepare" \
74     "$test2_command" \
75     --command-name "$test1_name" \
76     --conclude "$test1_conclude" \
77     --prepare "$test1_prepare" \
78     "$test1_command" \
79     --command-name "$test3_name" \
80     --conclude "$test3_conclude" \
81     --prepare "$test3_prepare" \
82     "$test3_command" \
83     #
84
85 $test1_conclude
86 $test2_conclude
87 $test3_conclude
```

LISTING C.1: Script Performance of the Compiler

Appendix D

Script Performance of the Output Binary

D.1 Script Performance of the Output Binary

```
1 #!/bin/sh
2
3 llvmproject_modified='https://github.com/Saverio976/llvm-improved-syntax-for-cxx-smart
   -pointer.git'
4
5 git clone "$llvmproject_modified" 'llvm-project-modified'
6
7 # llvm-project-modified
8 (
9     cur_path="$(pwd) "
10    cd llvm-project-modified
11    git checkout 'feat/improved_smart_pointer_syntax'
12    mkdir -p build
13    cd build
14    cmake "-DCMAKE_INSTALL_PREFIX=$cur_path/install" "-DLLVM_ENABLE_PROJECTS=clang" "-
       LLVM_TARGETS_TO_BUILD=X86" "-DCMAKE_BUILD_TYPE=Release" "../llvm"
15    make
16 )
17
18 CXX="llvm-project-modified/build/bin/clang++"
19
20 # 1 2 3 4 5 6 7 8 9 10
21
22 test1_name="new syntax - 1 2 3 4 5 6 7 8 9 10"
23 test1_command="./sort_algo_new.out 1 2 3 4 5 6 7 8 9 10"
```

```

24 $CXX -O2 "-smart-pointer=%,std::shared_ptr" ./sort_algo_new.cpp -o ./sort_algo_new.out
25
26 test2_name="current syntax - 1 2 3 4 5 6 7 8 9 10"
27 test2_command="./sort_algo_current.out 1 2 3 4 5 6 7 8 9 10"
28 $CXX -O2 ./sort_algo_current.cpp -o ./sort_algo_current.out
29
30 hyperfine \
31     --shell none \
32     --warmup 10000 \
33     --min-runs 10000 \
34     --command-name "$test1_name" \
35     "$test1_command" \
36     --command-name "$test2_name" \
37     "$test2_command" \
38     #
39
40 # random
41
42 random=$(echo -e 'import random\nfor _ in range(10000): print(int(random.uniform(1,
    1000)), end=" ")' | python)
43
44 test1_name="new syntax - random"
45 test1_command="./sort_algo_new.out $random"
46 $CXX -O2 "-smart-pointer=%,std::shared_ptr" ./sort_algo_new.cpp -o ./sort_algo_new.out
47
48 test2_name="current syntax - random"
49 test2_command="./sort_algo_current.out $random"
50 $CXX -O2 ./sort_algo_current.cpp -o ./sort_algo_current.out
51
52 hyperfine \
53     --shell none \
54     --warmup 5000 \
55     --min-runs 5000 \
56     --command-name "$test1_name" \
57     "$test1_command" \
58     --command-name "$test2_name" \
59     "$test2_command" \
60     #

```

LISTING D.1: Script Performance of the Output Binary

D.2 Sort Algo New Syntax

```

1 #include <cstdlib>
2 #include <iostream>
3 #include <memory>

```

```

4 #include <vector>
5
6 int partition(std::vector<int> %vec, int low, int high)
7 {
8     int pivot = vec->at(high);
9     int i = low - 1;
10
11     for (int j = low; j <= high - 1; j++) {
12         if (vec->at(j) <= pivot) {
13             i++;
14             std::swap(vec->at(i), vec->at(j));
15         }
16     }
17     std::swap(vec->at(i + 1), vec->at(high));
18     return i + 1;
19 }
20
21 void quick_sort(std::vector<int> %vec, int low, int high)
22 {
23     if (low < high) {
24         int index = partition(vec, low, high);
25         quick_sort(vec, low, index - 1);
26         quick_sort(vec, index + 1, high);
27     }
28 }
29
30 int main(int ac, char **av)
31 {
32     std::vector<int> %vec = std::make_shared<std::vector<int>>();
33
34     for (int i = 1; i < ac; i++) {
35         vec->push_back(std::atoi(av[i]));
36     }
37     quick_sort(vec, 0, vec->size() - 1);
38     for (const auto &i : *vec) {
39         std::cout << i << " ";
40     }
41     std::cout << std::endl;
42 }

```

LISTING D.2: Sort Algo New Syntax

D.3 Sort Algo Current Syntax

```

1 #include <cstdlib>
2 #include <iostream>

```

```
3 #include <memory>
4 #include <vector>
5
6 int partition(std::shared_ptr<std::vector<int>> vec, int low, int high)
7 {
8     int pivot = vec->at(high);
9     int i = low - 1;
10
11     for (int j = low; j <= high - 1; j++) {
12         if (vec->at(j) <= pivot) {
13             i++;
14             std::swap(vec->at(i), vec->at(j));
15         }
16     }
17     std::swap(vec->at(i + 1), vec->at(high));
18     return i + 1;
19 }
20
21 void quick_sort(std::shared_ptr<std::vector<int>> vec, int low, int high)
22 {
23     if (low < high) {
24         int index = partition(vec, low, high);
25         quick_sort(vec, low, index - 1);
26         quick_sort(vec, index + 1, high);
27     }
28 }
29
30 int main(int ac, char **av)
31 {
32     std::shared_ptr<std::vector<int>> vec = std::make_shared<std::vector<int>>();
33
34     for (int i = 1; i < ac; i++) {
35         vec->push_back(std::atoi(av[i]));
36     }
37     quick_sort(vec, 0, vec->size() - 1);
38     for (const auto &i : *vec) {
39         std::cout << i << " ";
40     }
41     std::cout << std::endl;
42 }
```

LISTING D.3: Sort Algo Current Syntax

References

- Chandler Carruth. 2019. There Are No Zero-cost Abstractions. In *CppCon 2019*. Youtube. <https://www.youtube.com/watch?v=rHlkrotSwcc>
- Clang. 2025. Clang: a C language family frontend for LLVM. <https://clang.llvm.org/index.html> Last accessed 01 april 2025.
- CppReference. 2024. cppreference. <https://en.cppreference.com/w/> Last accessed 29 mars 2025.
- Pedro Gonalo de Castro Correia. 2024. Towards a Memory Safe C with Rust-Inspired Checks.
- Slobodan Dmitrovic. 2020. *Modern C++ for Absolute Beginners: A Friendly Introduction to C++ Programming Language and C++11 to C++20 Standards* (1st edition ed.). Apress L. P, Berkeley, CA.
- Association for Computing Machinery. 2025. Chromium Docs - Checking out and building Chromium on Linux. https://chromium.googlesource.com/chromium/src/+/main/docs/linux/build_instructions.md Last accessed 08 april 2025.
- Free Software Foundation 2023. *Freeing after Malloc (The GNU C library)*. Free Software Foundation. https://www.gnu.org/software/libc/manual/html_node/Freeing-after-Malloc.html Last accessed 08 april 2025.
- GCC. 2025. GCC, the GNU Compiler Collection. <https://gcc.gnu.org/> Last accessed 04 april 2025.
- GNU. 2025. The GNU Operating System. <https://www.gnu.org/> Last accessed 04 april 2025.

- Jennifer Hamilton. 1997. Montana smart pointers: they’re smart, and they’re pointers. In *Proceedings of the 3rd Conference on USENIX Conference on Object-Oriented Technologies (COOTS) - Volume 3* (Portland, Oregon) (*COOTS’97*). USENIX Association, USA, 2.
- ISO. 1999. *BS ISO/IEC 14882:1998: Programming languages. C++*. British Standards Institute.
- ISO. 2001. *BS ISO/IEC 9899:1999: Programming languages. C*. British Standards Institute.
- ISO. 2011. *BS ISO/IEC 14882:2011: Information technology. Programming Languages. C++*. British Standards Institute.
- ISO. 2018. *BS ISO/IEC 14882:2017: Programming languages. C++*. British Standards Institute.
- Linda Jacobson. 2013. GCC’s move to C++. *LWN.net* (2013).
- Dennis Kafura. 2025. Reflections on McCabe’s Cyclomatic Complexity. *IEEE Transactions on Software Engineering* 51, 3 (2025), 700–705. <https://doi.org/10.1109/TSE.2025.3534580>
- LLVM. 2025. The LLVM Compiler Infrastructure. <https://llvm.org/index.html> Last accessed 01 april 2025.
- qBittorrent 2006. qBittorrent. <https://github.com/qbittorrent> Last accessed 08 august 2025.
- Snaipe. 2015. Implementing smart pointers for the C programming language. <https://snai.pe/posts/c-smart-pointers> Last accessed 08 august 2025.
- Bjarne Stroustrup. 1993. A history of C++: 1979–1991. *Sigplan Notices - SIGPLAN* 28 (03 1993), 271–297. <https://doi.org/10.1145/155360.155375>
- Herb Sutter. 2025. *Cpp2 and cppfront — An experimental ‘C++ syntax 2’ and its first compiler*. <https://hsutter.github.io/cppfront/welcome/overview/> Last accessed 06 april 2025.

- Sven van Haastregt and Anastasia Stulova. 2019. An overview of Clang. In *2019 LLVM Developers' Meeting*. Bash Films. <https://www.youtube.com/watch?v=5kkMpJpIGYU>
- Verysoft. 2025. Measurement of Halstead Metrics with Testwell CMT++ and CMT-Java (Complexity Measures Tool). https://www.verifysoft.com/en_halstead_metrics.html Last accessed 01 april 2025.